

by performing actions, making observations, and receiving rewards, all without prior knowledge of the source's dynamics, the potential usefulness of the updates to be queried, or the status of the update channels.

Within this framework, the hub takes an action $a(t) \in \mathcal{A}$ while the environment is in state $s(t) \in \mathcal{S}$ during the t -th slot. The action is passed to the environment, and after consulting the updated knowledge base, the hub observes the new state of the environment, $s(t+1)$. Based on this transition, the immediate net reward $r(t)$ resulting from the chosen action is computed. The recurring interaction between the hub and the environment over T_e steps is represented as a sequence $\langle s(t), a(t), r(t), s(t+1) \rangle_{t=0}^{T_e}$. Specifically, the reward $r(t)$ at the t -th slot is given by

$$r(t) = r_\mu(s(t), a(t), s(t+1)) \\ = v_{\text{cpt}}(\text{GoE}(t+1)) - \mu v_{\text{cpt}}^+(f_c(a(t))) \quad (18)$$

where μ is the Lagrange multiplier (see Section IV).

This formalism defines the state space $\mathcal{S}$, the action space $\mathcal{A}$, and the net rewards of the modeled CMDP in Section III-B. Leveraging the model-free approach, we develop a *learning-based* iterative algorithm to derive policies for solving the dual scheduling problem, i.e., $\hat{\mathcal{P}}$, from Section IV-A.

B. Learning-Based Iterative Algorithm

We adopt a similar iterative process to that outlined in Algorithm 1. The main distinction here is that we use model-free, learning-based solutions to find the class of effect-aware scheduling policies, π , within the inner loop.

1) *Computing π* : To derive the scheduling policies, we adapt two prominent on-policy DRL algorithms, *advantage actor-critic* (A2C) [35] and *proximal policy optimization* (PPO) [36], along with an off-policy algorithm, *Deep Q-network* (DQN) [37]. These algorithms are particularly well-suited for decision-making in high-dimensional spaces.

In each algorithm, the Q-function or the corresponding value function is computed based on the net reward defined in (18).

2) *Computing μ^** : We employ the same method as in Algorithm 1 to determine the optimal Lagrange multiplier for a given policy from the inner loop. In this context, the bisection search method is employed in the outer loop to iteratively and gradually identify the minimum Lagrange multiplier that meets the cost constraint of the scheduling problem $\mathcal{P}$, as formulated in (6).

VI. SIMULATION RESULTS

In this section, we evaluate the performance of the proposed model-based and model-free solutions outlined in Sections IV and V, respectively, within the context of effect-aware query scheduling. To assess their effectiveness, we compare these solutions against several well-established benchmark approaches.

A. Setup and Assumptions

We consider a system with $N = 4$ SAs observing a source characterized by $M = 2$ attributes and $K = 4$ AAs performing actions over $T = 1,000$ slots. The usefulness of an update on

TABLE I
PARAMETERS FOR SIMULATION RESULTS.

Parameter	Symbol	Value	Parameter	Symbol	Value
Number of slots	T	10^3	Parameters shaping $v_{\text{cpt}}(\cdot)$	α_{cpt}	0.5
Number of SAs	N	4		β_{cpt}	
Number of attributes	M	2		λ_{cpt}	2
Number of AAs	K	4	Refs. point	GoE_{ref}	0.2
Corr. observation probability	$p_{o,nm}, \forall n, m$	0.8	Cost per query	$f_c(1)$	0.5
Required attributes	$ \mathcal{M}_k , \forall k$	2	Cost flex. index	C_{flex}	0.75
Erasure probability	$p_{e,n}, \forall n$	0.2	Maximum AoI	Δ_{max}	4
Discount factor	γ	0.9	Converg. sens.	ϵ_π	10^{-6}
Shape parameters for $g_m(\cdot; \cdot), \forall m$	$\{\alpha_1, \alpha_2\}$	$\{0.5, 2\}$	Tolerance sens.	ϵ_μ	
	$\{\beta_1, \beta_2\}$	$\{0.5, 5\}$	Mixing factor	η	0.5
Size of usefulness	$ \mathcal{U} $	4	—	—	—

the m -th attribute is mapped to a value within the range $[0, 1]$, determined by applying

$$g_m(t; y_m(t)) = \min \left\{ 1, \frac{y_m^{\alpha_m-1}(t)(1 - y_m(t))^{\beta_m-1}}{B(\alpha_m, \beta_m)} \right\} \quad (19)$$

at the t -th slot, where $\alpha_m, \beta_m > 0, \forall m$, are shape parameters, and $B(\cdot, \cdot)$ is Beta function. Besides, we consider $\text{GoE}_m(t) = \frac{u_m(t)}{\Delta_m(t)}, \forall m$, and the CPT-based value function for an arbitrary $x \in \mathbb{R}$ is given by [12]

$$v_{\text{cpt}}(x) = \begin{cases} v_{\text{cpt}}^+(x) = (x - x_{\text{ref}})^{\alpha_{\text{cpt}}}, & x \geq x_{\text{ref}}; \\ v_{\text{cpt}}^-(x) = -\lambda_{\text{cpt}}(x_{\text{ref}} - x)^{\beta_{\text{cpt}}}, & x < x_{\text{ref}}, \end{cases} \quad (20)$$

with $\alpha_{\text{cpt}} = \beta_{\text{cpt}} = 0.5$, and $\lambda_{\text{cpt}} = 2$ being shape parameters under the given reference point x_{ref} . For simplicity and to facilitate comparison with non-probabilistic scheduling methods, we assume $w_{\text{cpt}}(x) = x, \forall x$.

Finally, the cost constraint is enforced by introducing a *cost flexibility index* C_{flex} multiplied by the discounted cumulative cost incurred from querying across *all* slots. Thus, we have

$$C_{\text{max}} = C_{\text{flex}} \left[\sum_{t=0}^{\infty} \gamma^t v_{\text{cpt}}^+(f_c(1)) \right] = C_{\text{flex}} \frac{v_{\text{cpt}}^+(f_c(1))}{1 - \gamma} \quad (21)$$

where $f_c(1)$ indicates the fixed cost per query. Unless stated otherwise, the default simulation parameter values are outlined in Table I.

To implement the DRL algorithms, namely A2C, PPO, and DQN, we follow the default hyperparameter settings as considered in their respective original papers, i.e., [35], [36], and [37]. However, we adjust the discount factor according to Table I. We employ the Adaptive Moment Estimation (Adam) optimizer for both DQN and PPO, whereas the Root Mean Square Propagation (RMSprop) optimizer is used for A2C. For each DRL algorithm, a model is trained over 100 episodes of interaction. Each episode comprises 10,000 time slots, generating a total of $T_e = 10^6$ environment steps. The key hyperparameters for these algorithms, along with their initial configurations, are summarized in Table II.